COMSATS University Islamabad, Lahore campus

COAL Mid Term Lab Exam

Submitted to: Sir Hamza Hamid

Submited by: Samiullah Ashfaq

SP24-BSE-107

Section A

Question 1:

MOV AX, 0107H ; Roll number value

MOV BX, 7010H ; Reverse roll number value

MOV CX, 0C042H

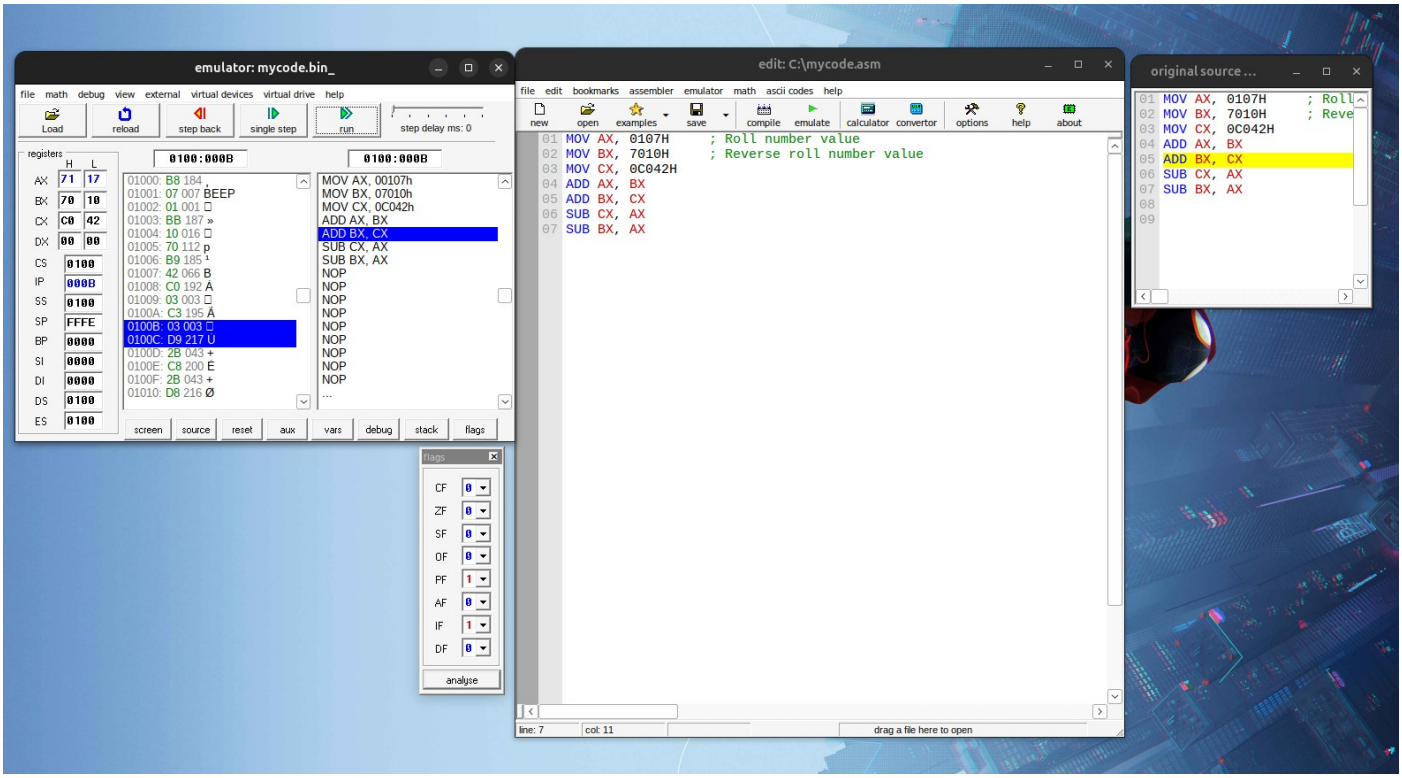
ADD AX, BX

ADD BX, CX

SUB CX, AX

SUB BX, AX

After 1st add:



Step-by-Step Execution

1st ADD: ADD AX, BX

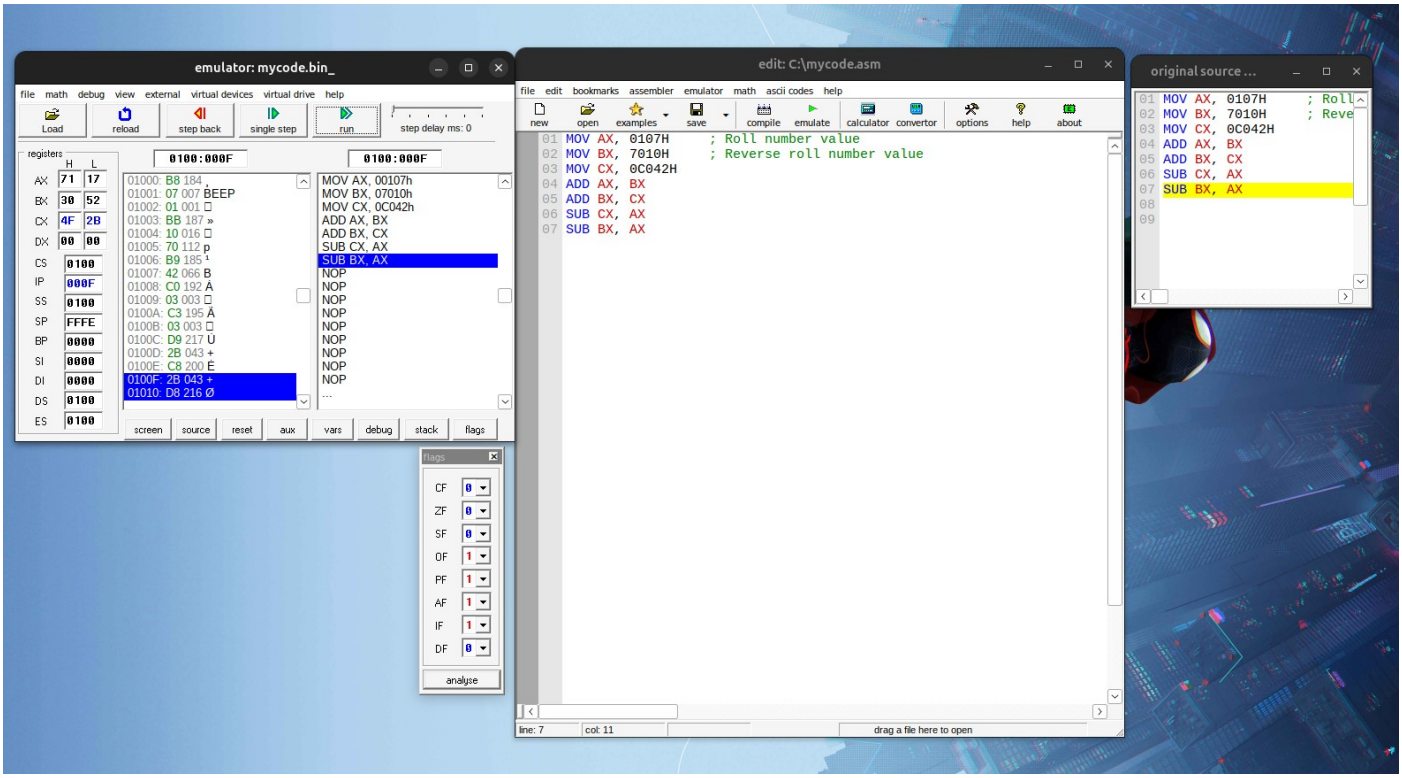
- **AX = 0107H + 7010H = 7117H**
- **AX = 7117H**
- **BX = 7010H**

Part (a) — After 1st ADD instruction

AX = 7117H

Flag	Value	Reason
CF	0	0107H + 7010H = 7117H which is ≤ FFFFh — result fits in 16 bits, no carry out of bit 15 → CF = 0
ZF	0	Result 7117H ≠ 0000H, so the Zero Flag is NOT set → ZF = 0
SF	0	MSB (bit 15) of 7117H = 0 (0111 0001 0001 0111), meaning the result is positive

After 1st sub:



Part (b) — After 1st SUB instruction

1st SUB: SUB CX, AX

- CX = 0C042H – 7117H = 4F2BH

Flag	Value	Reason
OF	1	The result (4F2BH) has an MSB of 0, meaning it is positive. This "sign flip" indicates a signed overflow occurred.
AF	1	he sum of the lowest 4 bits is 1011. Since a borrow was required in the original subtraction, AF is set to 1.

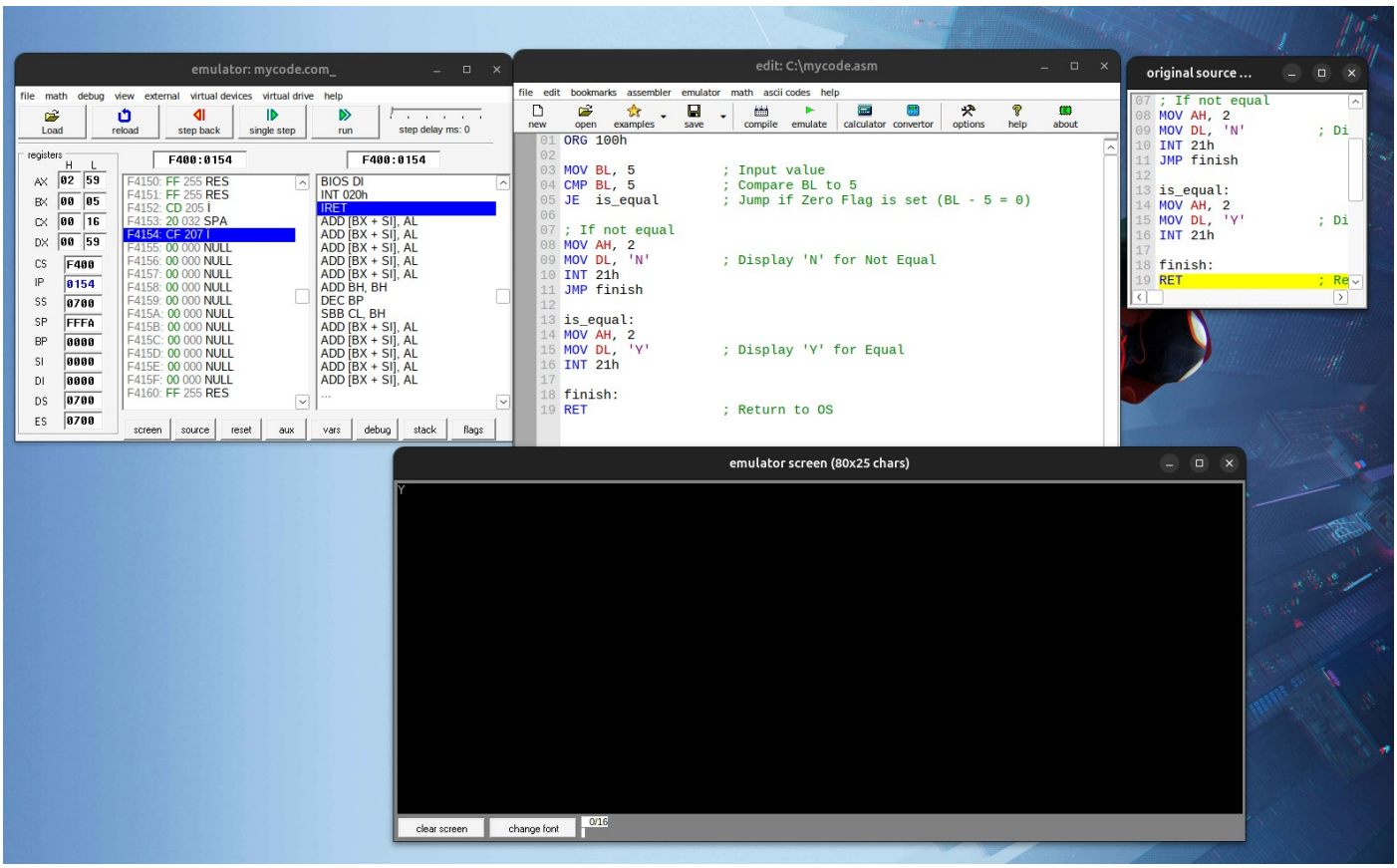
PF	**1**	The result is 4F2BH. The low byte is 2BH. Binary of 2BH: 0010 1011. Count the number of 1s: There are **four** 1s. Since 4 is an even number, the Parity Flag is set to 1 (Even Parity).

Question 2:

a. JE — Check if BL = 5

```
ORG 100h


MOV BL, 5          ; Input value

CMP BL, 5          ; Compare BL to 5

JE  is_equal       ; Jump if Zero Flag is set (BL - 5 = 0)


; If not equal

MOV AH, 2

MOV DL, 'N'        ; Display 'N' for Not Equal

INT 21h

JMP finish


is_equal:

MOV AH, 2

MOV DL, 'Y'        ; Display 'Y' for Equal

INT 21h


finish:
```

RET ; Return to OS



b. Print Y if CL = 20, else N

ORG 100h

MOV CL, 20 ; Input value

CMP CL, 20 ; Compare CL to 20

JE print_y ; If CL == 20, jump to print_y

; Else: Print 'N'

MOV AH, 2

MOV DL, 'N'

INT 21h

JMP exit_prog ; Jump over the 'Y' logic

print_y:

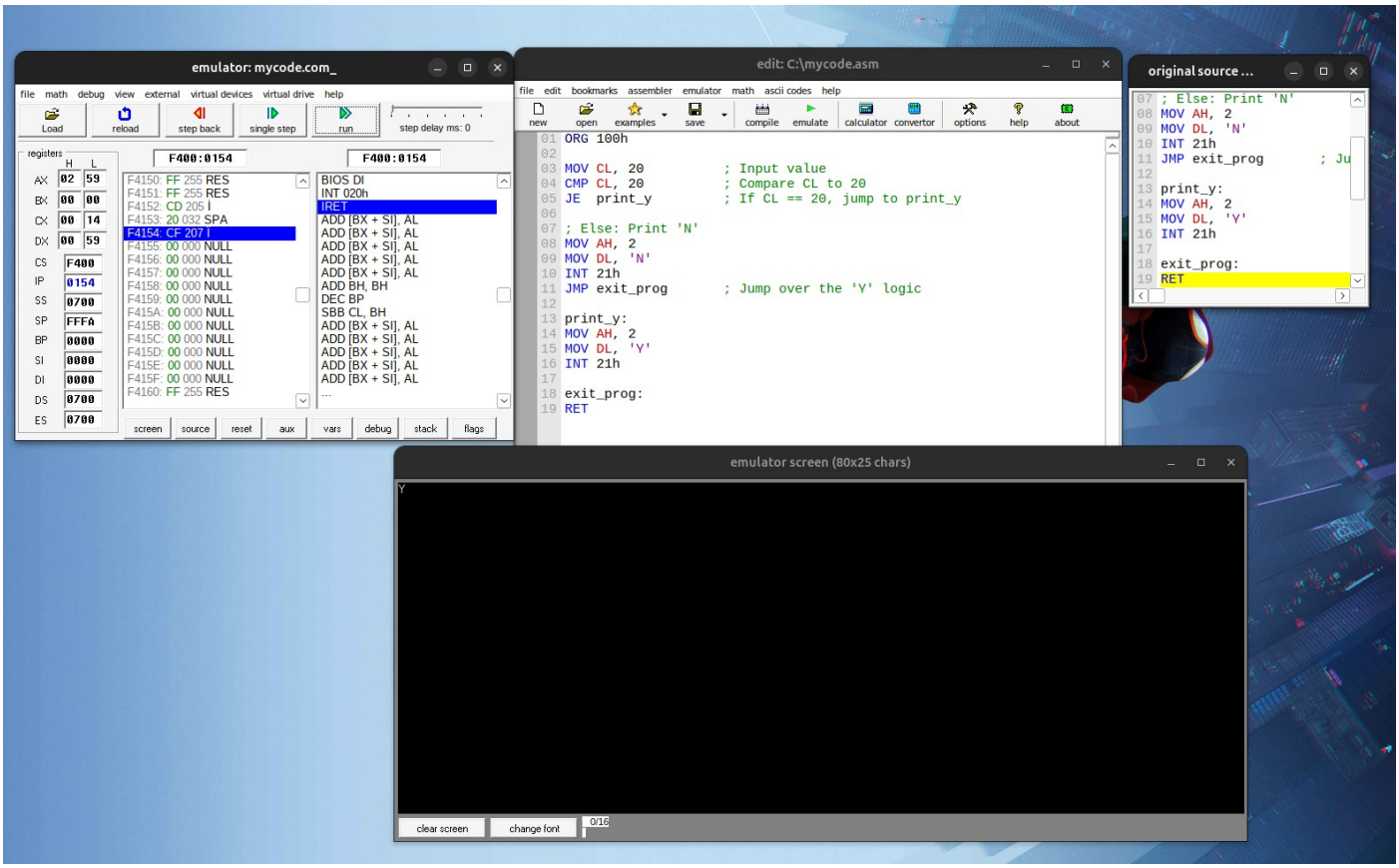
MOV AH, 2

MOV DL, 'Y'

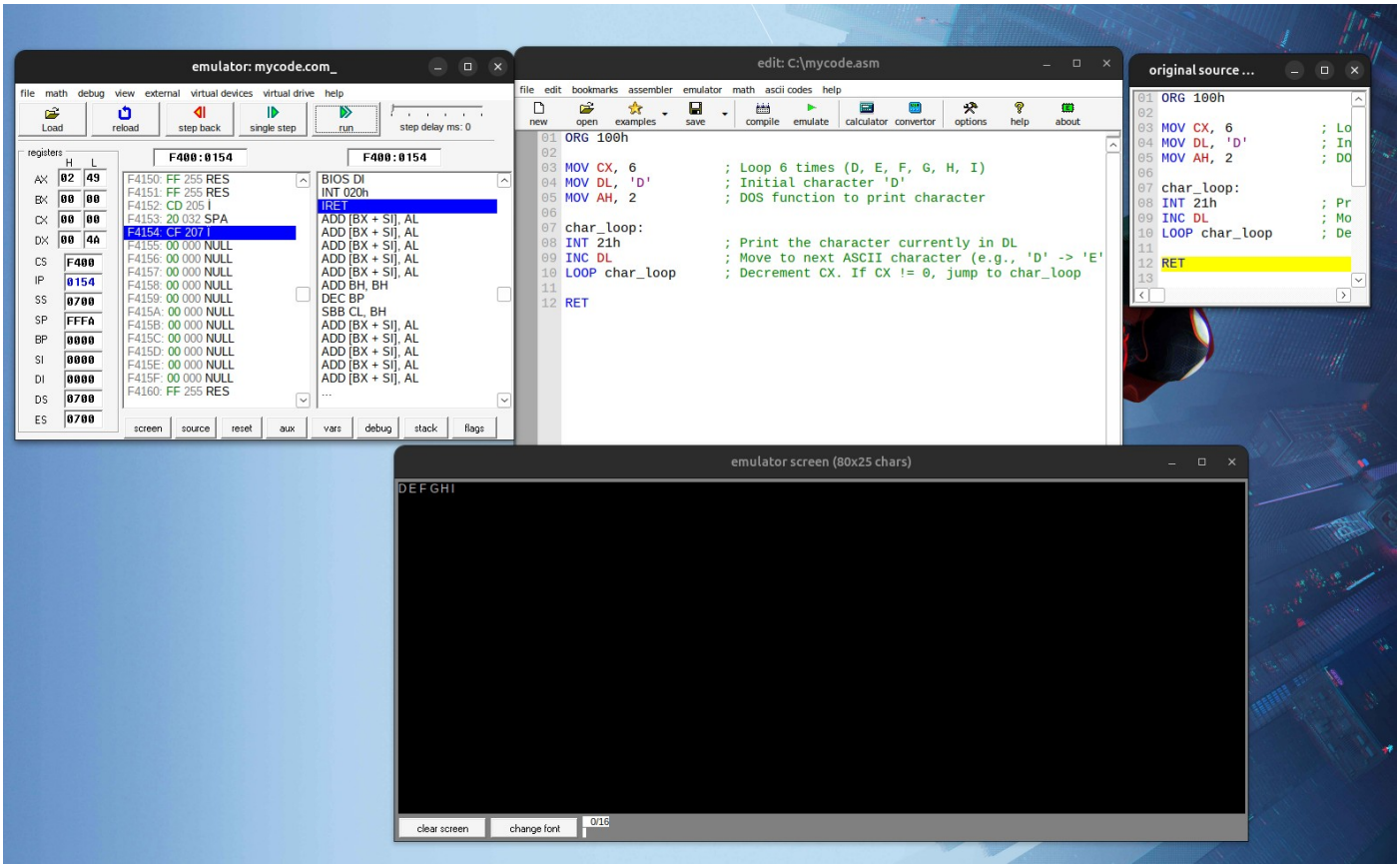
INT 21h

exit_prog:

RET



c. Display D to I using LOOP

ORG 100h

```
MOV CX, 6          ; Loop 6 times (D, E, F, G, H, I)

MOV DL, 'D'        ; Initial character 'D'

MOV AH, 2          ; DOS function to print character


char_loop:

INT 21h            ; Print the character currently in DL

INC DL             ; Move to next ASCII character (e.g., 'D' -> 'E')

LOOP char_loop     ; Decrement CX. If CX != 0, jump to char_loop


RET
```